API.md

Project: Arati Business ERP
Version: 0.1
Status: Draft / API Baseline
Audience: Backend developers, frontend developers, integration developers, QA, and technical reviewers

---

1. Purpose

This document defines the initial API contract and conventions for the ERP.

The API is intended to support:

- distributor operations
- wholesale sales
- retail/POS sales
- purchasing
- inventory
- stock transfers and adjustments
- customers, dealers, suppliers, and other parties
- payments and receivables
- sales and purchase returns
- delivery
- installation
- warranty
- audit
- TallyPrime integration
- authentication and authorization
- reporting/read-only queries

This is a **contract baseline**, not a final implementation specification. Business policies that have not yet been confirmed are explicitly marked `TBD — Business Confirmation Required`.

---

2. API Architecture

2.1 Style

The primary application API will use:

- REST
- JSON
- HTTPS
- versioned endpoints
- OpenAPI documentation
- stateless request handling
- PostgreSQL as the transactional source of truth

Proposed base URL:

/api/v1

Example:

GET /api/v1/products

2.2 API Responsibilities

The API layer should:

1. authenticate the caller
2. authorize the requested operation
3. validate input
4. execute domain/business rules
5. execute transactional database changes where required
6. create audit records for important mutations
7. return a predictable response
8. enqueue asynchronous work where appropriate
9. never expose internal database implementation details unnecessarily

The frontend must not be trusted to enforce business rules by itself.

---

3. General Conventions

3.1 HTTP Methods

Method	Purpose
GET	Read/query data
POST	Create resource or execute a domain action
PATCH	Partial update
PUT	Full replacement where genuinely required

DELETE	Reserved mainly for non-transactional resources; generally soft-delete/deactivate

For posted financial or stock transactions, `DELETE` should not be used to erase business history.

Use domain actions such as:

POST /sales/invoices/{id}/cancel
POST /purchase/invoices/{id}/cancel
POST /sales/returns/{id}/post
POST /stock/adjustments/{id}/post

---

4. Authentication

4.1 Proposed Mechanism

Authentication mechanism is **TBD — Technical Decision Required**.

The design must support:

- secure login
- access token/session management
- logout/revocation
- password reset
- optional MFA in a later phase
- user/session audit
- branch/organization context

4.2 Suggested Endpoints

POST /auth/login
POST /auth/logout
POST /auth/refresh
POST /auth/forgot-password
POST /auth/reset-password
GET /auth/me

Exact token strategy is TBD.

---

5. Authorization

Authorization should be role/permission based.

Example permissions:

products.read
products.create
products.update

purchases.read
purchases.create
purchases.post
purchases.cancel

sales.read
sales.create
sales.post
sales.cancel

payments.read
payments.create
payments.post

inventory.read
inventory.adjust
inventory.transfer
inventory.post

returns.sales.create
returns.sales.post
returns.purchase.create
returns.purchase.post

tally.read
tally.sync
tally.retry
tally.reconcile

reports.read
audit.read

The backend must verify permissions independently of frontend UI visibility.

---

6. Response Format

6.1 Success Response

For ordinary single-resource responses:

```
{
  "data": {
    "id": "uuid",
    "documentNumber": "SI-000123"
  }
}
```

For collections:

```
{
  "data": [],
  "meta": {
    "page": 1,
    "pageSize": 50,
    "total": 125
  }
}
```

6.2 Error Response

Standard format:

```
{
  "error": {
    "code": "CREDIT_LIMIT_EXCEEDED",
    "message": "Customer credit limit would be exceeded.",
    "details": {
      "customerId": "uuid",
      "availableCredit": "25000.00",
      "requiredCredit": "42000.00"
    },
    "requestId": "req_..."
  }
}
```

The API should not expose:

- SQL errors
- stack traces
- internal file paths
- secrets
- database schema details
- sensitive integration credentials

---

7. Validation

Validation occurs at multiple levels:

1. request schema validation
2. domain/business validation
3. database constraints

Example:

A sales invoice cannot be posted simply because the JSON is valid.

Posting must also verify:

- customer/party is valid
- branch is valid
- warehouse is valid where required
- products/variants exist
- quantities are valid
- prices/taxes/discounts are valid
- stock policy is satisfied
- credit policy is satisfied if applicable
- document status allows posting
- required serial/batch allocations are present
- duplicate posting is prevented

---

8. Idempotency

Idempotency is required for operations where a retry could otherwise create duplicate business transactions.

Especially:

- payment creation
- invoice posting
- return posting
- stock transfer posting
- Tally sync operations
- webhook/event processing where introduced

Client may send:

Idempotency-Key: 01J...

The backend should store enough information to safely recognize a repeated request.

Idempotency behavior and retention period are **TBD — Technical Decision Required**.

---

9. Pagination, Filtering, and Sorting

Collection endpoints should support a consistent query style.

Example:

GET /api/v1/products?page=1&pageSize=50
GET /api/v1/products?search=onida
GET /api/v1/sales/invoices?status=POSTED
GET /api/v1/sales/invoices?fromDate=2026-01-01&toDate=2026-01-31

Recommended parameters:

page
pageSize
search
sortBy
sortOrder
status
fromDate
toDate
branchId
warehouseId
partyId

Maximum `pageSize` should be enforced by the backend.

---

10. API Resource Groups

Proposed resource groups:

/auth
/users
/roles
/permissions

/organizations
/branches
/warehouses
/warehouse-locations

/brands
/categories
/units
/tax-rules
/products
/product-variants

/parties
/party-roles
/payment-terms

/price-lists
/party-price-lists

/purchases
/purchase-orders
/goods-receipts
/purchase-invoices

/sales
/quotations
/sales-orders
/sales-invoices

/inventory
/inventory-balances
/inventory-movements
/stock-transfers
/stock-adjustments

/payments

/sales-returns
/purchase-returns

/deliveries
/installations
/warranties

/tally
/audit-logs

/reports
/notifications
/attachments

---

11. Master Data APIs

11.1 Brands

GET /brands
GET /brands/{id}
POST /brands
PATCH /brands/{id}
POST /brands/{id}/deactivate

11.2 Categories

GET /categories
GET /categories/{id}
POST /categories
PATCH /categories/{id}
POST /categories/{id}/deactivate

11.3 Units

GET /units
GET /units/{id}
POST /units
PATCH /units/{id}

11.4 Tax Rules

GET /tax-rules
GET /tax-rules/{id}
POST /tax-rules
PATCH /tax-rules/{id}
POST /tax-rules/{id}/deactivate

Tax behavior and exact GST configuration are **TBD — Business Confirmation Required**.

11.5 Products

GET /products
GET /products/{id}
POST /products
PATCH /products/{id}
POST /products/{id}/deactivate

Possible filters:

brandId
categoryId

active
search
trackingType

11.6 Product Variants

GET /products/{productId}/variants
GET /product-variants/{id}
POST /products/{productId}/variants
PATCH /product-variants/{id}
POST /product-variants/{id}/deactivate

Variant data may include:

- SKU
- barcode
- unit
- dimensions
- weight
- MRP
- default pricing references
- tracking type
- tax rule

---

12. Party APIs

A generic party model should support customers, dealers, suppliers, and distributors without creating separate duplicated master systems.

12.1 Parties

GET /parties
GET /parties/{id}
POST /parties
PATCH /parties/{id}
POST /parties/{id}/deactivate

Filters:

role
search
active
city
creditStatus

12.2 Party Roles

GET /party-roles
POST /parties/{id}/roles
DELETE /parties/{id}/roles/{role}

12.3 Payment Terms

GET /payment-terms
GET /payment-terms/{id}
POST /payment-terms
PATCH /payment-terms/{id}

Credit limit behavior is **TBD — Business Confirmation Required**.

---

13. Pricing APIs

13.1 Price Lists

GET /price-lists
GET /price-lists/{id}
POST /price-lists
PATCH /price-lists/{id}
POST /price-lists/{id}/deactivate

13.2 Price List Items

GET /price-lists/{id}/items
POST /price-lists/{id}/items
PATCH /price-lists/{id}/items/{itemId}
DELETE /price-lists/{id}/items/{itemId}

13.3 Party Pricing

GET /parties/{id}/price-lists
POST /parties/{id}/price-lists
DELETE /parties/{id}/price-lists/{priceListId}

The pricing engine should determine the effective price rather than relying on frontend calculations.

---

14. Purchase APIs

14.1 Purchase Orders

GET /purchase-orders
GET /purchase-orders/{id}
POST /purchase-orders
PATCH /purchase-orders/{id}
POST /purchase-orders/{id}/submit
POST /purchase-orders/{id}/approve
POST /purchase-orders/{id}/cancel

Typical lifecycle:

DRAFT
→ SUBMITTED
→ APPROVED
→ PARTIALLY_RECEIVED
→ RECEIVED

Exact approval and status rules are **TBD — Business Confirmation Required**.

14.2 Goods Receipts

GET /goods-receipts
GET /goods-receipts/{id}
POST /goods-receipts
PATCH /goods-receipts/{id}
POST /goods-receipts/{id}/post
POST /goods-receipts/{id}/cancel

Posting a goods receipt may create inventory movements.

It must not be possible to post the same receipt twice.

For tracked inventory, the API must support serial/batch allocation.

14.3 Purchase Invoices

GET /purchase-invoices
GET /purchase-invoices/{id}
POST /purchase-invoices
PATCH /purchase-invoices/{id}
POST /purchase-invoices/{id}/post
POST /purchase-invoices/{id}/cancel

A purchase invoice may reference one or more goods receipts depending on the confirmed workflow.

The final relationship should be validated against the ERD and actual business process before implementation.

---

15. Sales APIs

15.1 Quotations

GET /quotations
GET /quotations/{id}
POST /quotations
PATCH /quotations/{id}
POST /quotations/{id}/send
POST /quotations/{id}/accept
POST /quotations/{id}/cancel

15.2 Sales Orders

GET /sales-orders
GET /sales-orders/{id}
POST /sales-orders
PATCH /sales-orders/{id}
POST /sales-orders/{id}/confirm
POST /sales-orders/{id}/cancel

Potential lifecycle:

DRAFT
→ CONFIRMED
→ PARTIALLY_FULFILLED
→ FULFILLED

Stock reservation is **TBD — Business Confirmation Required**.

15.3 Sales Invoices

GET /sales-invoices
GET /sales-invoices/{id}
POST /sales-invoices
PATCH /sales-invoices/{id}
POST /sales-invoices/{id}/post
POST /sales-invoices/{id}/cancel

The same sales engine should support:

- retail/POS

- wholesale
- dealer sales
- other approved sales channels

Do not create a separate database flow for POS.

A specialized convenience endpoint may be added later:

POST /pos/sales

Whether this is needed is **TBD — Technical Decision Required**.

---

16. Inventory APIs

16.1 Inventory Balances

GET /inventory/balances
GET /inventory/balances/{variantId}

Filters:

warehouseId
locationId
variantId
batchId

Inventory balance is a performance projection.

The authoritative history is the inventory movement ledger.

16.2 Inventory Movements

GET /inventory/movements
GET /inventory/movements/{id}

This is primarily read-only for normal users.

Users should not directly edit historical inventory movements.

16.3 Stock Transfers

GET /stock-transfers
GET /stock-transfers/{id}
POST /stock-transfers
PATCH /stock-transfers/{id}
POST /stock-transfers/{id}/post

POST /stock-transfers/{id}/cancel

A posted transfer must atomically:

1. reduce source inventory
2. increase destination inventory
3. record corresponding movement history

Concurrency must be handled so two users cannot consume the same available stock.

16.4 Stock Adjustments

GET /stock-adjustments
GET /stock-adjustments/{id}
POST /stock-adjustments
PATCH /stock-adjustments/{id}
POST /stock-adjustments/{id}/post
POST /stock-adjustments/{id}/cancel

Adjustment reason should be mandatory.

Approval requirements are **TBD — Business Confirmation Required**.

---

17. Serial and Batch APIs

17.1 Serial Units

GET /serial-units
GET /serial-units/{id}
GET /serial-units/{id}/history

Serial units should generally be created/changed through business transactions rather than arbitrary CRUD operations.

17.2 Batches

GET /batches
GET /batches/{id}
POST /batches
PATCH /batches/{id}

Batch creation/allocation behavior should be tied to the relevant inventory receipt process.

---

18. Payment APIs

18.1 Payments

GET /payments
GET /payments/{id}
POST /payments
PATCH /payments/{id}
POST /payments/{id}/post
POST /payments/{id}/cancel

18.2 Payment Allocation

GET /payments/{id}/allocations
POST /payments/{id}/allocations
DELETE /payments/{id}/allocations/{allocationId}

The system should support:

- one payment → many invoices
- many payments → one invoice
- partial invoice payment
- advance payment

Overpayment behavior is **TBD — Business Confirmation Required**.

---

19. Sales Return APIs

GET /sales-returns
GET /sales-returns/{id}
POST /sales-returns
PATCH /sales-returns/{id}
POST /sales-returns/{id}/post
POST /sales-returns/{id}/cancel

A sales return should capture:

- original sale reference where available
- party
- returned items
- quantities
- serial/batch allocations
- return reason
- disposition

Possible disposition values:

RESELLABLE
REPAIR_REQUIRED
DAMAGED
SCRAP
SUPPLIER_RETURN

The final disposition workflow is **TBD — Business Confirmation Required**.

---

20. Purchase Return APIs

GET /purchase-returns
GET /purchase-returns/{id}
POST /purchase-returns
PATCH /purchase-returns/{id}
POST /purchase-returns/{id}/post
POST /purchase-returns/{id}/cancel

Purchase returns should support:

- supplier
- original purchase reference where available
- items
- quantities
- batch/serial references
- return reason

---

21. Delivery APIs

GET /deliveries
GET /deliveries/{id}
POST /deliveries
PATCH /deliveries/{id}
POST /deliveries/{id}/dispatch
POST /deliveries/{id}/deliver
POST /deliveries/{id}/cancel

Potential lifecycle:

PLANNED
→ DISPATCHED
→ DELIVERED

Whether delivery requires separate item-level allocation is **TBD — Business Confirmation Required**.

---

22. Installation APIs

GET /installations
GET /installations/{id}
POST /installations
PATCH /installations/{id}
POST /installations/{id}/schedule
POST /installations/{id}/complete
POST /installations/{id}/cancel

Installation should support:

- customer/site
- assigned technician
- appointment
- completion status
- notes
- relevant product/serial reference

Technician management depth is **TBD — Scope Decision Required**.

---

23. Warranty APIs

GET /warranties
GET /warranties/{id}
POST /warranties
PATCH /warranties/{id}

Warranty data may reference:

- customer
- product/variant
- serial unit where applicable
- sale
- installation
- start/end dates
- warranty status

Warranty start-date policy is **TBD — Business Confirmation Required**.

---

24. Tally Integration APIs

Tally is an external accounting system.

The ERP must remain operational if Tally synchronization is temporarily unavailable.

24.1 Connections

GET /tally/connections
GET /tally/connections/{id}
POST /tally/connections
PATCH /tally/connections/{id}
POST /tally/connections/{id}/test

Connection credentials must never be returned in ordinary GET responses.

24.2 Mappings

GET /tally/mappings
POST /tally/mappings
PATCH /tally/mappings/{id}

Mappings may include:

- ERP party → Tally ledger
- ERP product → Tally stock item
- ERP tax rule → Tally tax configuration
- ERP warehouse/branch → Tally location where applicable
- ERP document type → Tally voucher type

Exact mappings are **TBD — Business Confirmation Required**.

24.3 Sync

GET /tally/sync-jobs
GET /tally/sync-jobs/{id}
POST /tally/sync-jobs/{id}/retry
POST /tally/sync-jobs/{id}/reconcile

Sync should be:

- asynchronous where appropriate
- retryable
- idempotent
- auditable

- observable
- reconcilable

The exact Tally communication mechanism is **TBD — Technical Decision Required**.

---

25. Audit APIs

GET /audit-logs
GET /audit-logs/{id}

Audit logs should capture important actions such as:

- login/security events
- master changes
- posting/cancellation
- payment actions
- inventory adjustments
- stock transfers
- returns
- Tally operations
- permission changes

Normal users should not be able to modify audit records.

---

26. Reports APIs

Reports should initially be read-only APIs.

Possible endpoints:

GET /reports/sales-summary
GET /reports/purchase-summary
GET /reports/stock-summary
GET /reports/stock-movement
GET /reports/receivables
GET /reports/payables
GET /reports/customer-ledger
GET /reports/supplier-ledger
GET /reports/profitability
GET /reports/tally-reconciliation

The first report set should be selected based on actual operational requirements rather than building every possible report.

Large reports may need asynchronous export jobs.

---

27. Notifications APIs

GET /notifications
POST /notifications/{id}/read
POST /notifications/read-all

Notifications may cover:

- low stock
- pending approvals
- overdue receivables
- failed Tally sync
- delivery status
- installation schedule
- warranty events

Notification scope should remain limited to useful operational alerts.

---

28. Attachments APIs

GET /attachments/{id}
POST /attachments
DELETE /attachments/{id}

Attachments may be linked to:

- parties
- products
- purchase documents
- sales documents
- returns
- deliveries
- installations
- warranty records

Files should be stored outside PostgreSQL where appropriate, with metadata in the database.

---

29. Document Numbering

Business documents require human-readable numbers.

Examples:

SI-000001
PI-000001
SO-000001
PO-000001
SR-000001
PR-000001
PAY-000001

The exact numbering format, prefix, branch sequencing, fiscal-year behavior, and concurrency strategy are **TBD — Business Confirmation Required / Technical Design Required**.

Document numbers must not be generated in a way that allows duplicate numbers under concurrent requests.

---

30. Transactional Actions

Business actions should be represented explicitly.

Examples:

POST /sales-invoices/{id}/post
POST /sales-invoices/{id}/cancel

POST /goods-receipts/{id}/post

POST /stock-transfers/{id}/post

POST /payments/{id}/post

POST /sales-returns/{id}/post

A posting endpoint should:

1. verify current status
2. verify permissions

3. validate business rules
4. lock/validate relevant records where necessary
5. perform all required database changes in one transaction
6. create inventory/accounting/audit records as appropriate
7. commit atomically
8. return the resulting state

If any required step fails, the transaction should roll back.

---

31. Concurrency

The API must assume multiple users may act on the same data simultaneously.

Examples:

- two users selling the last unit
- transfer and sale happening simultaneously
- two payments being allocated to the same invoice
- two users editing the same draft
- duplicate post requests

The backend should use appropriate PostgreSQL transaction isolation, row locks, optimistic concurrency, unique constraints, or other mechanisms.

Exact concurrency strategy is **TBD — Technical Design Required** and should be finalized with the physical schema.

---

32. Optimistic Concurrency

For important editable documents, a version field may be used.

Example request:

```
{
  "customerId": "uuid",
  "version": 4
}
```

If the database version is already 5, the API should reject the stale update rather than silently overwriting another user's work.

Exact implementation is **TBD — Technical Decision Required**.

---

33. Auditability

For important mutations, the API should retain:

- actor/user
- timestamp
- organization
- branch
- action
- resource type
- resource ID
- relevant before/after information
- request ID where useful

Audit data must be append-oriented and protected from ordinary modification.

---

34. API Security

Minimum requirements:

- HTTPS in production
- secure password storage
- authorization on every protected endpoint
- input validation
- rate limiting on authentication endpoints
- request size limits
- safe file upload validation
- protection against injection
- protection against broken object-level authorization
- secrets kept outside source code
- sensitive fields excluded from logs
- security headers at the application/gateway layer

Security implementation details belong in `SECURITY.md`.

---

35. API Logging and Observability

Every request should have a correlation/request ID.

Example:

X-Request-ID: req_01J...

Logs should help trace:

request
→ API handler
→ domain action
→ database transaction
→ async job
→ external integration

Do not log:

- passwords
- access tokens
- Tally credentials
- payment secrets
- unnecessary personal data

---

36. Async Jobs

Asynchronous processing may be used for:

- Tally synchronization
- report exports
- notifications
- large file processing
- other long-running operations

The system should not introduce a queue solely because it is fashionable.

Redis/BullMQ or another queue implementation remains a technical decision based on actual requirements.

---

37. API Versioning

Initial version:

/api/v1

Breaking API changes should use a new major API version where necessary.

Non-breaking additions should generally remain within the current version.

---

38. OpenAPI

The API should be documented using OpenAPI.

The generated specification should include:

- endpoints
- request schemas
- response schemas
- authentication
- error codes
- pagination
- examples
- enum values

Swagger UI or equivalent documentation may be exposed in development and protected/non-public in production as appropriate.

---

39. Common Error Codes

Initial error vocabulary:

VALIDATION_ERROR
UNAUTHORIZED
FORBIDDEN
NOT_FOUND

DUPLICATE_RESOURCE
INVALID_STATUS_TRANSITION
CONCURRENT_MODIFICATION

INSUFFICIENT_STOCK
SERIAL_NOT_AVAILABLE
BATCH_NOT_AVAILABLE

CREDIT_LIMIT_EXCEEDED
PAYMENT_ALLOCATION_INVALID

DOCUMENT_ALREADY_POSTED
DOCUMENT_ALREADY_CANCELLED

TALLY_CONNECTION_FAILED
TALLY_SYNC_FAILED
TALLY_MAPPING_MISSING

IDEMPOTENCY_CONFLICT
RATE_LIMITED
INTERNAL_ERROR

The final error catalog should be maintained centrally in the backend.

---

40. API Contract Example

Example sales invoice creation:

POST /api/v1/sales-invoices
Content-Type: application/json
Idempotency-Key: 01JEXAMPLE

Request:

```
{
  "partyId": "uuid",
  "branchId": "uuid",
  "warehouseId": "uuid",
  "items": [
    {
      "productVariantId": "uuid",
      "quantity": "2",
      "unitPrice": "25000.00",
      "discount": "1000.00",
      "taxRuleId": "uuid"
    }
  ],
  "payment": {
    "mode": "CASH",
    "amount": "49000.00"
  }
}
```

Response:

```
{
  "data": {
    "id": "uuid",
```

```
    "documentNumber": "SI-000123",
    "status": "DRAFT",
    "grandTotal": "49000.00"
  }
}
```

Posting:

POST /api/v1/sales-invoices/{id}/post

Response:

```
{
  "data": {
    "id": "uuid",
    "documentNumber": "SI-000123",
    "status": "POSTED"
  }
}
```

The exact request/response schema will be finalized after the ERD and business workflow validation.

---

41. What the API Must Not Do

The API should not:

- let frontend code directly modify stock balances
- allow arbitrary edits to posted invoices
- silently delete financial history
- depend synchronously on Tally for normal sales/purchases
- expose database tables as an unvalidated CRUD interface
- duplicate business logic across frontend and backend
- trust client-calculated totals without server verification
- allow duplicate posting through retries
- allow unauthorized branch/warehouse access
- expose secrets through API responses

---

42. API Implementation Order

Recommended implementation order:

Phase 1 — Foundation

/auth
/users
/roles
/permissions
/organizations
/branches
/warehouses

Phase 2 — Masters

/brands
/categories
/units
/tax-rules
/products
/product-variants
/parties
/payment-terms
/price-lists

Phase 3 — Procurement

/purchase-orders
/goods-receipts
/purchase-invoices

Phase 4 — Inventory

/inventory/balances
/inventory/movements
/stock-transfers
/stock-adjustments
/serial-units
/batches

Phase 5 — Sales

/quotations
/sales-orders
/sales-invoices

Phase 6 — Money

/payments
/payment allocations
/receivables

Phase 7 — Returns / Fulfillment

/sales-returns

/purchase-returns
/deliveries
/installations
/warranties

Phase 8 — Integration / Reporting

/tally
/reports
/notifications
/audit
/attachments

This sequence can change after workflow validation.

---

43. Open API Decisions

The following must not be guessed:

1. Authentication/token strategy
2. Exact authorization model
3. API rate limits
4. Idempotency retention
5. Document numbering strategy
6. Fiscal-year numbering behavior
7. Credit-limit enforcement
8. Negative-stock policy
9. GST/tax calculation rules
10. Invoice posting rules
11. Payment/advance/overpayment behavior
12. Stock reservation behavior
13. Serial/batch allocation mechanics
14. Delivery item model
15. Installation workflow
16. Warranty start rules
17. Tally communication mechanism
18. Tally voucher/master mapping
19. Report/export architecture
20. Exact concurrency strategy

All unresolved business policies should be confirmed before the affected endpoint is treated as final.

---

44. Definition of API Done

The API baseline is considered implementation-ready only when:

- endpoint ownership is defined
- request/response schemas are defined
- business actions are defined
- authorization is defined
- error codes are defined
- transaction boundaries are defined
- idempotency behavior is defined for critical operations
- concurrency behavior is defined for stock/money operations
- OpenAPI specification can be generated
- integration contracts are defined
- tests cover critical business paths

---

45. Next Artifacts

Recommended next documents:

1. `ERD.md` — validate database relationships
2. `BUSINESS-WORKFLOWS.md` — confirm real purchase/sale/inventory/payment workflows
3. `TALLY-INTEGRATION.md` — define accounting integration contract
4. `SECURITY.md` — security architecture and controls
5. `PHASES.md` — implementation roadmap
6. `TESTING.md` — test strategy
7. OpenAPI specification — after business workflows and ERD are validated

Important: Do not freeze every endpoint or schema in this document until the ERD and actual business workflows have been reviewed.